CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 7

Name: Hari Krishan **UID:** 24BAI70898

Section/Group: 24AIT_NTPP 3 (A) **Branch/Semester:** CSE_AIML (CS221) / 5th **Subject Code:** 24 CSP-337 **Subject Name:** Full Stack - II **Date of Performance:** **Submitted to:** Mr.Rakshit Ajay

Experiment – 7.1

1) AIM: To implement secure backend APIs using JWT Authentication and role-based authorization.

2) OBJECTIVE

1. To understand JWT-based authentication
2. To implement authentication filters
3. To configure Spring Security for API protectio
4. To enforce role-based access control using RBAC

3) SOFTWARE REQUIREMENT

1. Java(JDK 17+)
2. Spring Boot
3. IDE(VSCode/Eclipse)
4. Postman/API testing tool
5. Mysql workbench

4) THEORY

In modern backend systems, securing APIs is essential to prevent unauthorized access and protect sensitive data. **JWT (JSON Web Token)** is widely used for stateless authentication, where the client includes a token in each request instead of relying on server-side sessions.

Spring Security provides a flexible framework to implement authentication and authorization. It uses a **security filter chain** to intercept requests, validate credentials, and enforce access rules.

R**ole-Based Access Control (RBAC)** is used to restrict access based on user roles such as ADMIN or USER. This ensures that users can only access resources permitted to their role. Method-level security using annotations like @PreAuthorize allows fine-grained control over endpoint access. ogether, JWT and RBAC enable:

- **Stateless authentication**
- **Scalable backend systems**
  - **Secure API access control**

5) IMPLEMENTATION/PROCEDURE

1. Configure Spring security
2. Implement JWT Authorization filter
3. Secure endpoints using authentication rules
4. Apply role-based restrictions using @PreAuthorize
5. Ensure to create the project using dependencies: Spring Web, Spring Security, Spring Data JPA, MySQL Driver, Validation and JJWT

6) CODE

Create a database in mysql workbench: CREATE DATABASE jwt_security_db;

Update application.properties spring.application.name=jwt-security-api

spring.datasource.url=jdbc:mysql://localhost:3306/jwt_security_api_db
spring.datasource.username=root
spring.datasource.password=MyPassword spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
server.port=8080 **update the pom.xml by adding the JWT dependencies**

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
```

Create user entity(User.java) package

```
com.example.jwt_security_api.entity;

import jakarta.persistence.*;
@Entity
@Table(name="users") public
class User {
    @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
   @Column(unique = true, nullable = false, length=50)
   private String username;
   @Column(nullable = false, length=100)
private String password;
   @Column(nullable = false, length=50)
   private String role;
   public User() {
   }
   public User(String username, String password, String role) {
this.username = username;        this.password = password;
      this.role = role;
   }
   public Long getId() {
return id;
   }
   public String getUsername() {
      return username;
   }
   public String getPassword() {
      return password;
   }
   public String getRole() {
      return role;
   }
   public void setId(Long id) {
this.id = id;
   }
   public void setUsername(String username) {
      this.username = username;
   }
   public void setPassword(String password) {
this.password = password;
   }
   public void setRole(String role) {
      this.role = role;
   }
```

```
}
```

Create User.Repository.java

```
package com.example.jwt_security_api.repository; import
com.example.jwt_security_api.entity.User; import
org.springframework.data.jpa.repository.JpaRepository; import
java.util.Optional;
public interface UserRepository extends JpaRepository<User, Long> {
Optional<User> findByUsername(String username);     boolean
existsByUsername(String username);
    }
```

Create CustomeUserDetailService.java to connect spring security to database

```
package com.example.jwt_security_api.service; import
com.example.jwt_security_api.entity.User;
import com.example.jwt_security_api.repository.UserRepository; import
org.springframework.security.core.authority.SimpleGrantedAuthority; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.security.core.userdetails.UserDetailsService; import
org.springframework.security.core.userdetails.UsernameNotFoundException; import
org.springframework.stereotype.Service;
import java.util.Collections;
@Service
public class CustomUserDetailsService implements UserDetailsService  {
   private final UserRepository userRepository;
   public CustomUserDetailsService(UserRepository userRepository) {
this.userRepository = userRepository;
   }
   @Override
   public UserDetails loadUserByUsername(String username)
throws UsernameNotFoundException {
      User user = userRepository.findByUsername(username)
            .orElseThrow(() ->
                  new UsernameNotFoundException("User not found:"+ username));
return new org.springframework.security.core.userdetails.User(
user.getUsername(),              user.getPassword(),
Collections.singletonList(
                  new SimpleGrantedAuthority("ROLE_" + user.getRole())
            )
      );
   }
}
```

Create SecurityConfig.java for password hashing and authentication package

```
com.example.jwt_security_api.config;

import com.example.jwt_security_api.security.JwtAuthenticationFilter; import
com.example.jwt_security_api.service.CustomUserDetailsService; import
```

```
org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
@Configuration
public class SecurityConfig {
   private final JwtAuthenticationFilter jwtAuthenticationFilter;
private final CustomUserDetailsService userDetailsService;     public
SecurityConfig(
            JwtAuthenticationFilter jwtAuthenticationFilter,
       CustomUserDetailsService userDetailsService) {
    this.jwtAuthenticationFilter = jwtAuthenticationFilter;
      this.userDetailsService = userDetailsService;
   }
@Bean
   public PasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
   }
@Bean
   public DaoAuthenticationProvider authenticationProvider() {
DaoAuthenticationProvider  provider  =                          new
DaoAuthenticationProvider(userDetailsService);
provider.setPasswordEncoder(passwordEncoder());
      return provider;
   }
@Bean
   public AuthenticationManager authenticationManager(
AuthenticationConfiguration configuration)
         throws Exception {
      return configuration.getAuthenticationManager();
   }
@Bean
   public SecurityFilterChain securityFilterChain(
HttpSecurity http)
         throws Exception {
http
            .csrf(csrf -> csrf.disable())
.sessionManagement(session ->
session.sessionCreationPolicy(
                     SessionCreationPolicy.STATELESS
               )
```

```
            )
            .authorizeHttpRequests(auth -> auth
                  // Public endpoints
                  .requestMatchers(
                        "/api/auth/**"
                  ).permitAll()
                  // Frontend entry point and its static assets
                  .requestMatchers(
    "/",
                        "/index.html",
                        "/styles.css",
                        "/app.js"
                  ).permitAll()
                 // ADMIN only
                  .requestMatchers(
                        "/api/admin/**"
                  ).hasRole("ADMIN")
// USER and ADMIN
                  .requestMatchers(
                        "/api/users/**"
                  ).hasAnyRole("USER", "ADMIN")
                  // Everything else
                  .anyRequest().authenticated()
            )
            .authenticationProvider(
                  authenticationProvider()
            )
            .addFilterBefore(
                  jwtAuthenticationFilter,
                  UsernamePasswordAuthenticationFilter.class
            );
      return http.build();
   } }
```

Create JwtService.java: Generate JWT access tokens and validate their signature, subject, and expiry. store username, issue time(1hour).

```
package com.example.jwt_security_api.security;
import io.jsonwebtoken.Claims; import
io.jsonwebtoken.Jwts; import
io.jsonwebtoken.io.Decoders; import
io.jsonwebtoken.security.Keys;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service; import
javax.crypto.SecretKey; import
```

```
java.nio.charset.StandardCharsets; import java.util.Date; import
java.util.HashMap; import java.util.Map;
@Service
public class JwtService {
   private final String SECRET_KEY =
         "mySuperSecretKeyForJwtAuthentication123456789";
private SecretKey getSigningKey() {        return
Keys.hmacShaKeyFor(
            SECRET_KEY.getBytes(StandardCharsets.UTF_8)
      );
   }
   public String generateToken(UserDetails userDetails) {
   Map<String, Object> claims = new HashMap<>();
   return Jwts.builder()
            .claims(claims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date())
            .expiration(
                  new Date(System.currentTimeMillis() + 1000 * 60 * 60)
            )
            .signWith(getSigningKey())
            .compact();
   }
   private Claims extractAllClaims(String token) {
return Jwts.parser()
            .verifyWith(getsigningKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
   }
   public boolean isTokenValid(
         String token,
         UserDetails userDetails) {
      String username = extractUsername(token);
return username.equals(userDetails.getUsername());
   }
}
```

Create JwtAuthenticationFilter.java to read the `Bearer` token, validate it, load its user, and set the authenticated user in Spring Security's context.

```
package com.example.jwt_security_api.security;
import com.example.jwt_security_api.service.CustomUserDetailsService;
import jakarta.servlet.FilterChain; import
jakarta.servlet.ServletException; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.servlet.http.HttpServletResponse;
```

```
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder; import
org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component; import
org.springframework.web.filter.OncePerRequestFilter; import java.io.IOException;
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter{ private
   final JwtService jwtService;
   private final CustomUserDetailsService
   userDetailsService; public JwtAuthenticationFilter(
   JwtService jwtService,
         CustomUserDetailsService userDetailsService) {
   this.jwtService = jwtService;
   this.userDetailsService = userDetailsService;
   }
   @Override
   protected void doFilterInternal(
HttpServletRequest request,
         HttpServletResponse response,
FilterChain filterChain)          throws
ServletException, IOException {       String
authHeader =
request.getHeader("Authorization");       if
(authHeader == null ||
            !authHeader.startsWith("Bearer ")) {
         filterChain.doFilter(request, response);
         return;
}
      String token =
authHeader.substring(7);
      try {
         String username =
               jwtService.extractUsername(token);
if (username != null &&
SecurityContextHolder
                     .getContext()
                     .getAuthentication() == null) {
UserDetails userDetails =
userDetailsService
                        .loadUserByUsername(username);
if (jwtService.isTokenValid(                    token,
                  userDetails)) {
               UsernamePasswordAuthenticationToken
                     authentication =
                     new UsernamePasswordAuthenticationToken(
userDetails,
                           null,
```

```
                        userDetails.getAuthorities()
                    );
                authentication.setDetails(
                        new WebAuthenticationDetailsSource()
                            .buildDetails(request)
                );
                SecurityContextHolder
                        .getContext()
                        .setAuthentication(authentication);
            }
          }
        } catch (Exception e) {
          System.out.println(
                "Invalid JWT token: " + e.getMessage()
          );
        }
        filterChain.doFilter(request, response);
    }
}
```

Create RegisterRequest.java: Define and validate registration input.

```
package com.example.jwt_security_api.dto; import
jakarta.validation.constraints.NotBlank; public
class RegisterRequest {
   @NotBlank     private String
username;     @NotBlank
private String password;
public String getUsername() {
return username;
   }
   public void setUsername(String username) {
      this.username = username;
   }
   public String getPassword() {
return password;
   }
   public void setPassword(String password) {
this.password = password;
   }
}
```

Create LoginRequest.java: Define and validate login input.

```
package com.example.jwt_security_api.dto; import
jakarta.validation.constraints.NotBlank; public class LoginRequest {
@NotBlank      private String username;      @NotBlank      private String
password;     public String getUsername() {        return username;
   }
```

```
    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
    return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

Create AuthResponse.java: define login response

```
package com.example.jwt_security_api.dto;
public class AuthResponse {     private
String token;     public
AuthResponse(String token) {
        this.token = token;
    }
    public String getToken() {
        return token;
    }
    public void setToken(String token) {
        this.token = token;
    }
}
```

Create AuthController.java: Implement registration and login

```
package com.example.jwt_security_api.controller; import
com.example.jwt_security_api.dto.AuthResponse; import
com.example.jwt_security_api.dto.LoginRequest; import
com.example.jwt_security_api.dto.RegisterRequest; import
com.example.jwt_security_api.entity.User; import
com.example.jwt_security_api.repository.UserRepository; import
com.example.jwt_security_api.security.JwtService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity; import
org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.core.Authentication; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/auth") public
class AuthController {
```

```
    private final UserRepository userRepository;     private final
PasswordEncoder passwordEncoder;     private final
AuthenticationManager authenticationManager;
    private final JwtService jwtService;
public AuthController(
            UserRepository userRepository,
            PasswordEncoder passwordEncoder,
            AuthenticationManager authenticationManager,
            JwtService jwtService,
            ) {
        this.userRepository = userRepository;
       this.passwordEncoder = passwordEncoder;
      this.authenticationManager = authenticationManager;
       this.jwtService = jwtService;
    }

    @PostMapping("/register")
    public ResponseEntity<String> register(
            @Valid @RequestBody RegisterRequest request) {
if (userRepository
                .findByUsername(request.getUsername())
                .isPresent()) {
return ResponseEntity
                    .badRequest()
                    .body("Username already exists");
        }
        User user = new User();
        user.setUsername(request.getUsername());
        user.setPassword(
passwordEncoder.encode(
                        request.getPassword()
                )
        );
        user.setRole("USER");
userRepository.save(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(
                "User registered successfully"
        );
    }
    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
            @Valid @RequestBody LoginRequest request) {
Authentication authentication =
authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(
                        request.getUsername(),
```

```
                request.getPassword()
            )
        );
    UserDetails userDetails =
        (UserDetails) authentication.getPrincipal();
    User user = userRepository.findByUsername(userDetails.getUsername())
        .orElseThrow();
String accessToken =
        jwtService.generateToken(userDetails);
  }
}
```

Create UserController.java: add enpoints restricted to authenticated users

```
package com.example.jwt_security_api.controller; import
org.springframework.security.core.Authentication; import
org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/users") public
class UserController {
@GetMapping("/profile")
  public String profile(Authentication authentication) {
return "Welcome USER: "
        + authentication.getName();
  }
  @GetMapping("/dashboard")
public String dashboard() {
    return "User dashboard accessed successfully";
  }
}
```

Create AdminContoller.java: add admin-only endpoints

```
package com.example.jwt_security_api.controller; import
com.example.jwt_security_api.dto.UserResponse; import
com.example.jwt_security_api.repository.UserRepository; import
org.springframework.http.ResponseEntity; import
org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController @RequestMapping("/api/admin") public
class AdminController {   private final UserRepository
userRepository;   public AdminController(UserRepository
userRepository) {
    this.userRepository = userRepository;
  }
  @GetMapping("/dashboard")   public
String dashboard() {      return "Welcome
to Admin Dashboard";
  }
  @GetMapping("/users")   public
List<UserResponse> users() {      return
```

```
userRepository.findAll().stream()
.map(user -> new UserResponse(
user.getId(),
user.getUsername(),
user.getRole()))
            .toList();
   }
   @DeleteMapping("/users/{id}")    public
ResponseEntity<Void> deleteUser(
@PathVariable Long id) {        if
(!userRepository.existsById(id)) {
return ResponseEntity.notFound().build();
      }
      userRepository.deleteById(id);
      return ResponseEntity.noContent().build();
   } }
```

7) OUTPUT

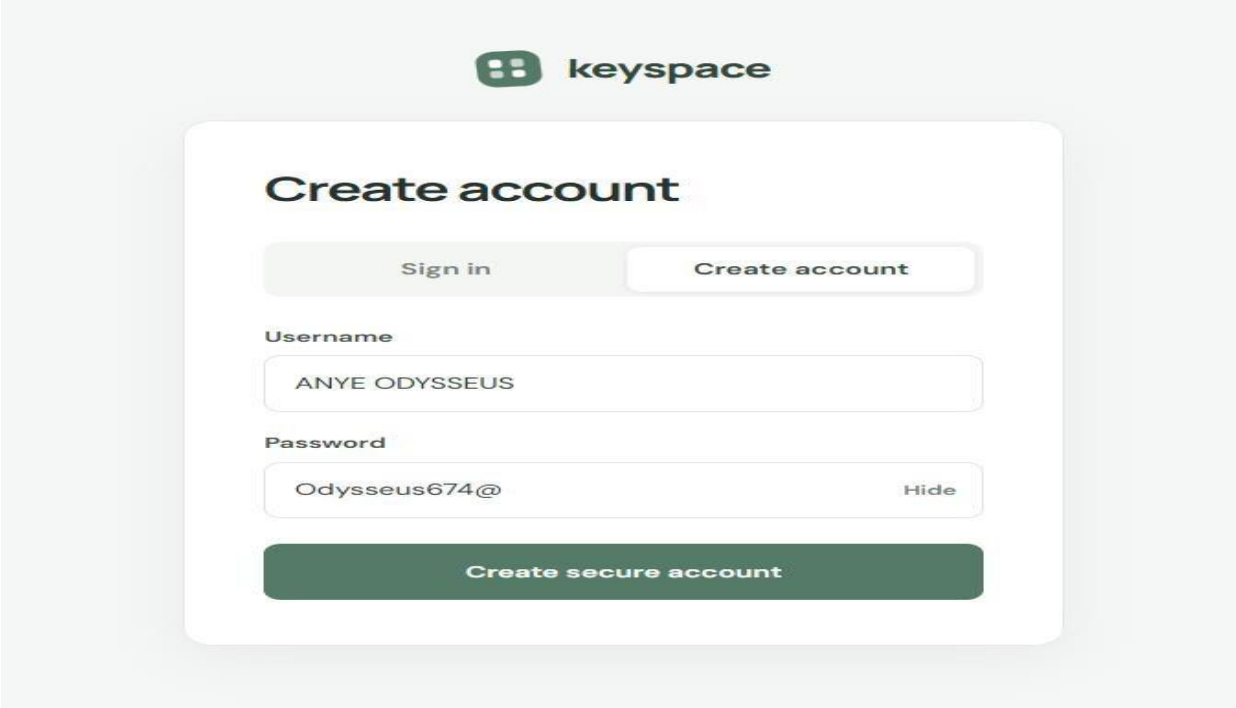


Fig 7.1.2: login(signin) or Create a new user(account)

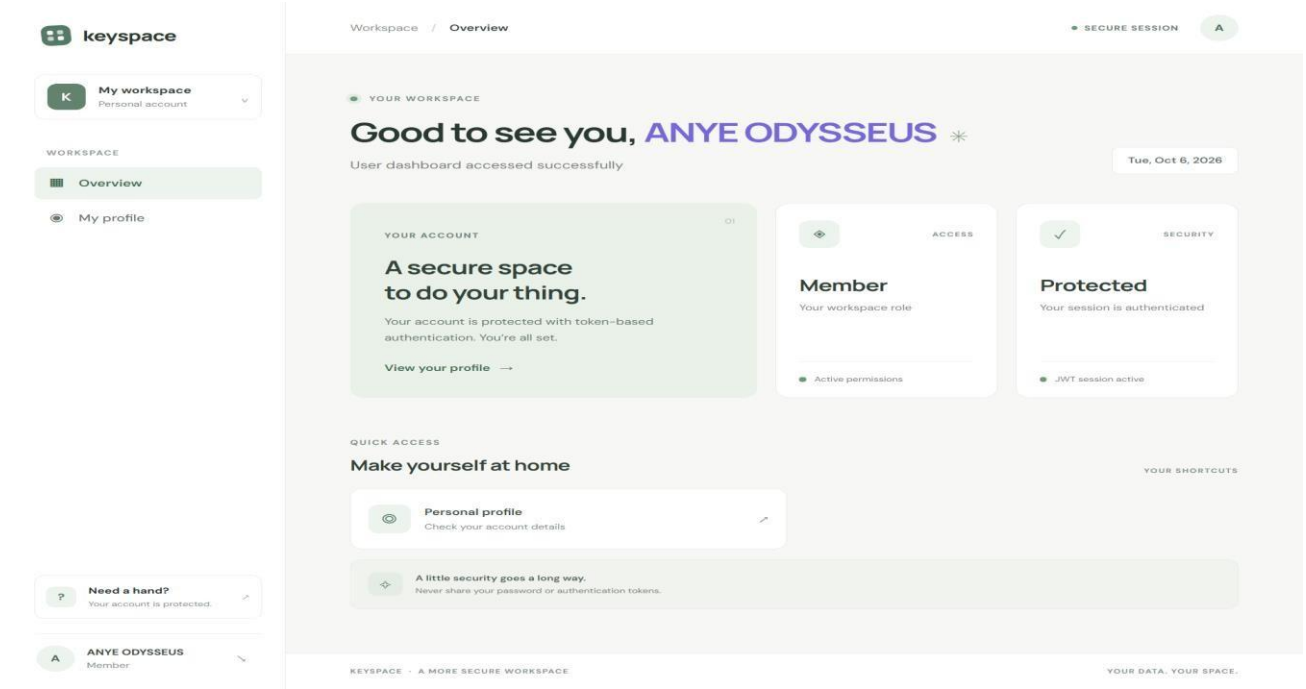


Fig 7.1.2: login in user dashboard with valid token

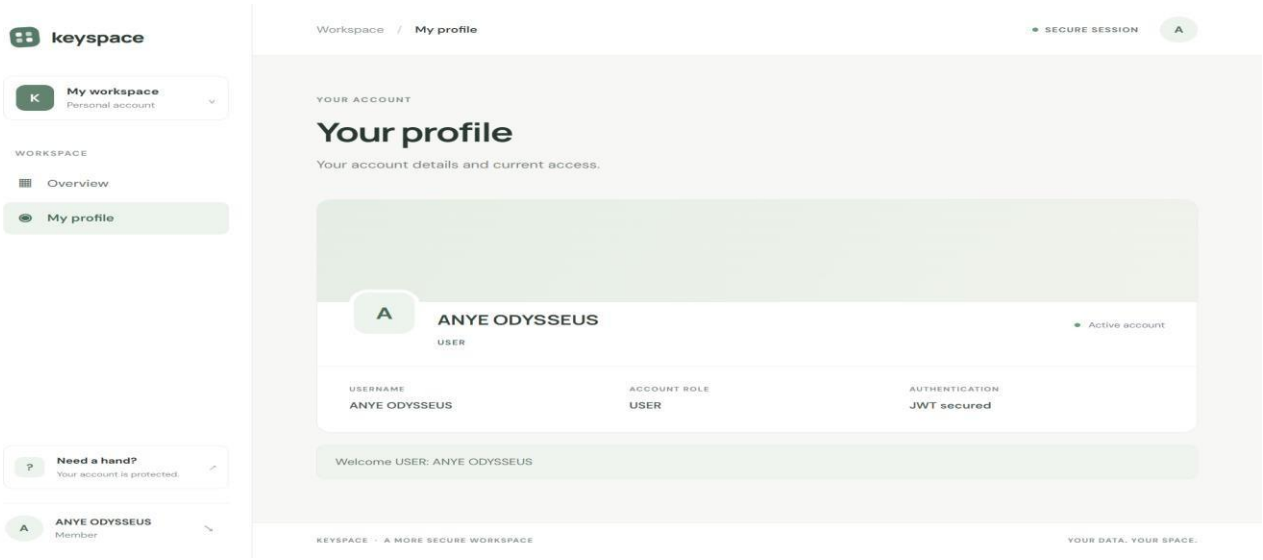


Fig7.1.3: login user profile

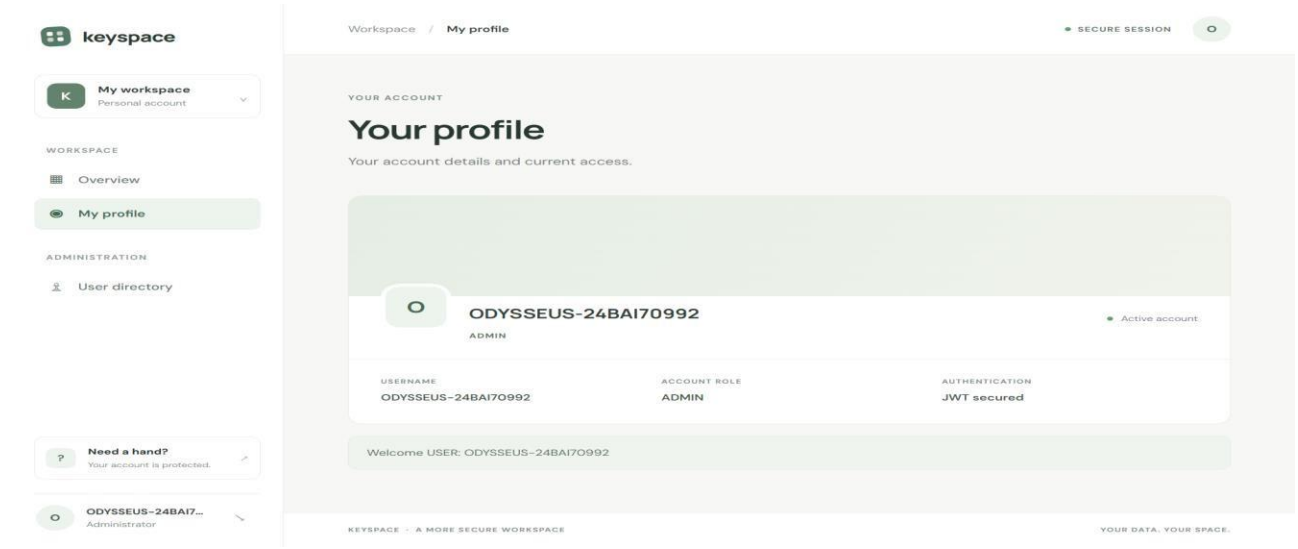


Fig 7.1.4: Admin profile

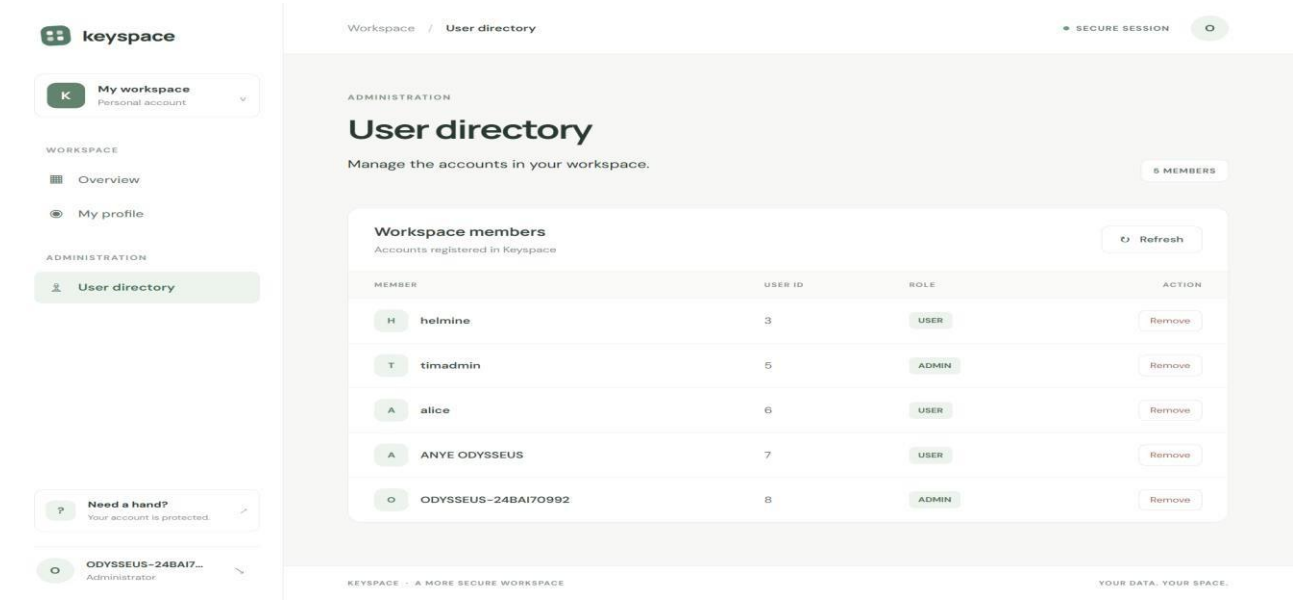


Fig 7.1.5:An admin can manage users(view, delete users)

8) LEARNING OUTCOMES

- Explain how Spring Boot, Spring Security, and JPA work together in a secured backend.
- Design a user entity and repository for storing account details and roles.
- Validate registration and login input using request DTOs and Jakarta Bean Validation.
- Hash passwords with BCrypt and explain why passwords should be hashed rather than reversibly encrypted.
- Generate and validate signed JWT access tokens, including subject and expiration claims.

- Implement a request filter that reads bearer tokens and sets the authenticated user in Spring Security's context.
- Configure stateless, role-based access control and test public, authenticated, and role-restricted endpoints.

Experiment – 7.2

1) AIM: To implement encryption and secure storage mechanisms for protecting sensitive data and credentials.

2) OBJECTIVE

1. To understand encryption techniques
2. To securely store third-party credentials
3. To implement AES-based data encryption
4. To manage token lifecycle using access and refresh tokens

3) SOFTWARE REQUIREMENT

1. Spring Boot
2. Database(MYSQL/PostgreSQL)
3. Java Cryptography libraries

4) THEORY

In secure backend systems, protecting sensitive information such as OAuth tokens and credentials is critical. Storing such data in plaintext exposes the system to security risks. Therefore, encryption techniques are used to ensure data confidentiality. **AES (Advanced Encryption Standard)** is a widely used symmetric encryption algorithm that securely encrypts data before storage and decrypts it when needed.

In addition to encryption, managing authentication tokens is essential. A common strategy involves:

- Access tokens(short-lived) for API access
- Refresh tokens(long-lived) for obtaining new access tokens

This approach enhances security by limiting the exposure of access tokens and enabling controlled session management.

5) IMPLEMENTATION/PROCEDURE

1. Implement AES encryption for sensitive data
2. Encrypt and store OAuth tokens securely
3. Implement access and refresh token strategy
4. Handle token refresh and rotation

6) CODE

Add configurations in applications.properties for the JWT signing key, access-token lifetime, and refreshtoken lifetime. Supply secrets through environment variables rather than hard-coding them.

```
security.jwt.secret-base64=${JWT_SECRET_BASE64}
security.jwt.access-token-ttl=PT15M
security.encryption.aes-key-base64=${AES_KEY_BASE64} security.refresh-token.ttl-days=7
```

update JWT generation in Jwtservice.java to use the configured signing key and access-token expiry.

```
package com.example.jwt_security_api.security; import
io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts; import
io.jsonwebtoken.io.Decoders; import
io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service; import
javax.crypto.SecretKey; import java.util.Date; import
java.time.Duration;
import java.time.Instant;
@Service public class JwtService {
private final SecretKey signingKey;
private final Duration accessTokenTtl;
public JwtService(
        @Value("${security.jwt.secret-base64}") String secretBase64,
@Value("${security.jwt.access-token-ttl}") Duration accessTokenTtl) {        this.signingKey =
Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretBase64));        this.accessTokenTtl =
accessTokenTtl;
   }
  public String generateToken(UserDetails userDetails) {
      Instant now = Instant.now();

      return Jwts.builder()
            .subject(userDetails.getUsername())
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plus(accessTokenTtl)))
```

```
            .signWith(signingKey)
            .compact();
    }

    public String extractUsername(String token) {
        return extractAllClaims(token).getSubject();
    }
    public boolean isTokenValid(String token, UserDetails userDetails) {
Claims claims = extractAllClaims(token);
        return claims.getSubject().equals(userDetails.getUsername())
            && claims.getExpiration().after(new Date());
    }
    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(signingKey)
            .parseSignedClaims(token)
            .getPayload();
     }
}
```

Create RefreshToken.java(entity) to store the user, token hash,expiry and revoked status package

```
com.example.jwt_security_api.entity;

import jakarta.persistence.*; import
java.time.Instant;
@Entity
@Table (name = "refresh_tokens") public
class RefreshToken {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id", nullable = false)     private
User user;
    @Column(nullable = false, unique = true, length = 64)
private String tokenHash;     @Column(nullable = false)
private Instant expiresAt;     @Column(nullable = false)
private boolean revoked;
    protected RefreshToken() {
    }
    public RefreshToken(User user, String tokenHash, Instant expiresAt) {
this.user = user;         this.tokenHash = tokenHash;         this.expiresAt =
expiresAt;
        this.revoked = false;
    }
    public Long getId() {
return id;
    }
```

```
    public User getUser() {
return user;
    }
    public String getTokenHash() {
return tokenHash;
    }
    public Instant getExpiresAt() {
        return expiresAt;
    }
    public boolean isRevoked() {
return revoked;
    }
    public void setRevoked(boolean revoked) {
        this.revoked = revoked;
    }
}
```

Create RefreshTokenRepository.java to add token persistence and locking

```
package com.example.jwt_security_api.repository; import
com.example.jwt_security_api.entity.RefreshToken; import
jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock; import
org.springframework.data.jpa.repository.Query; import
org.springframework.data.repository.query.Param; import
java.util.Optional;
public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("""
            select token
            from RefreshToken token
where token.tokenHash = :tokenHash
              and token.revoked = false
            """)
    Optional<RefreshToken> findActiveForUpdate(@Param("tokenHash") String tokenHash);

}
```

Create RefreshTokenService.java: It returns a random raw token to the client but stores only its SHA-256 hash

```
package com.example.jwt_security_api.service; import
com.example.jwt_security_api.entity.RefreshToken; import
com.example.jwt_security_api.entity.User;
import com.example.jwt_security_api.repository.RefreshTokenRepository;
import org.springframework.beans.factory.annotation.Value; import
org.springframework.http.HttpStatus; import
org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional; import
org.springframework.web.server.ResponseStatusException;
import java.nio.charset.StandardCharsets; import
java.security.MessageDigest; import
java.security.NoSuchAlgorithmException; import
java.security.SecureRandom;
import java.time.Instant;
import java.util.Base64; import
java.util.HexFormat;
@Service
public class RefreshTokenService {
   private final RefreshTokenRepository refreshTokenRepository;
private final long ttlDays;
   private final SecureRandom secureRandom = new SecureRandom();
public RefreshTokenService(
         RefreshTokenRepository refreshTokenRepository,
         @Value("${security.refresh-token.ttl-days}") long ttlDays) {
this.refreshTokenRepository = refreshTokenRepository;        this.ttlDays
= ttlDays;
   }
   @Transactional
   public String issue(User user) {
      String rawToken = createRawToken();
refreshTokenRepository.save(
            new RefreshToken(
                  user,
                  hashToken(rawToken),
                  Instant.now().plusSeconds(ttlDays * 24 * 60 * 60)
            )
      );
      return rawToken;
   }
   @Transactional
   public Rotation rotate(String rawToken) {
RefreshToken storedToken = refreshTokenRepository
            .findActiveForUpdate(hashToken(rawToken))
.orElseThrow(RefreshTokenService::invalidToken);        if
(!storedToken.getExpiresAt().isAfter(Instant.now())) {
         throw invalidToken();
      }
      storedToken.setRevoked(true);
refreshTokenRepository.save(storedToken);        String
replacement = issue(storedToken.getUser());
      return new Rotation(storedToken.getUser(), replacement);
   }
   @Transactional
   public void revoke(String rawToken) {
refreshTokenRepository
```

```
            .findActiveForUpdate(hashToken(rawToken))
            .ifPresent(token -> {
token.setRevoked(true);
refreshTokenRepository.save(token);
            });
    }
    private String createRawToken() {
byte[] bytes = new byte[32];
secureRandom.nextBytes(bytes);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
    }
    private String hashToken(String rawToken) {
try {
          byte[] digest = MessageDigest.getInstance("SHA-256")
                .digest(rawToken.getBytes(StandardCharsets.UTF_8));
    return HexFormat.of().formatHex(digest);     } catch
    (NoSuchAlgorithmException e) {
          throw new IllegalStateException("SHA-256 is unavailable", e);
        }
    }
    private static ResponseStatusException invalidToken() {
        return new ResponseStatusException(
            HttpStatus.UNAUTHORIZED,
             "Invalid or expired refresh token"
        );
    }
    public record Rotation(User user, String refreshToken) {
    }
}
```

Create RefreshRequest.java package

```
com.example.jwt_security_api.dto;        import
jakarta.validation.constraints.NotBlank;
public record RefreshRequest( @NotBlank String refreshToken) {
}
```

Update authentication response in AuthResponse.java to include both access and refresh tokens

```
package com.example.jwt_security_api.dto; public class AuthResponse {     private String token;     private
String refreshToken;
    public AuthResponse(String token, String refreshToken) {
this.token = token;
        this.refreshToken = refreshToken;
    }
    public String getToken() {
        return token;
    }
    public void setToken(String token) {
```

```
        this.token = token;
    }
    public String getRefreshToken() {
        return refreshToken;
    }
    public void setRefreshToken(String refreshToken) {
this.refreshToken = refreshToken;
    }
}
```

Update AuthController.java to issue a refresh token on login, add POST(/refresh) torotate it, add POST(logout) to revoke it

```
package com.example.jwt_security_api.controller; import
com.example.jwt_security_api.dto.AuthResponse; import
com.example.jwt_security_api.dto.LoginRequest; import
com.example.jwt_security_api.dto.RefreshRequest; import
com.example.jwt_security_api.dto.RegisterRequest;
import com.example.jwt_security_api.entity.User;
import com.example.jwt_security_api.repository.UserRepository;
import com.example.jwt_security_api.security.JwtService; import
com.example.jwt_security_api.service.RefreshTokenService; import
jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.core.Authentication; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.security.core.userdetails.UserDetailsService; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/auth") public
class AuthController {
    private final UserRepository userRepository;     private final
PasswordEncoder passwordEncoder;     private final
AuthenticationManager authenticationManager;     private final
JwtService jwtService;
    private final RefreshTokenService refreshTokenService;
private final UserDetailsService userDetailsService;
    public AuthController(
            UserRepository userRepository,
            PasswordEncoder passwordEncoder,
            AuthenticationManager authenticationManager,
            JwtService jwtService,
            RefreshTokenService refreshTokenService,
UserDetailsService userDetailsService) {
this.userRepository = userRepository;
```

```
this.passwordEncoder = passwordEncoder;
this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
        this.refreshTokenService = refreshTokenService;
        this.userDetailsService = userDetailsService;
    }
    @PostMapping("/register")
    public ResponseEntity<String> register(
            @Valid @RequestBody RegisterRequest request) {
if (userRepository
                .findByUsername(request.getUsername())
                .isPresent()) {
return ResponseEntity
                    .badRequest()
                    .body("Username already exists");
        }
        User user = new User();
        user.setUsername(request.getUsername());
        user.setPassword(
    passwordEncoder.encode(
                        request.getPassword()
                )
        );
       user.setRole("USER");
    userRepository.save(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(
                "User registered successfully"
        );
    }
    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
            @Valid @RequestBody LoginRequest request) {
Authentication authentication =
authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(
                        request.getUsername(),
                        request.getPassword()
                    )
                );
        UserDetails userDetails =
                (UserDetails) authentication.getPrincipal();
        User user = userRepository.findByUsername(userDetails.getUsername())
                .orElseThrow();
String accessToken =
```

```
            jwtService.generateToken(userDetails);
String refreshToken = refreshTokenService.issue(user);
return ResponseEntity.ok(
            new AuthResponse(accessToken,refreshToken)
      );
   }
   @PostMapping("/refresh")
   public ResponseEntity<AuthResponse> refresh(
         @Valid @RequestBody RefreshRequest request) {
RefreshTokenService.Rotation rotation =
            refreshTokenService.rotate(request.refreshToken());
UserDetails userDetails = userDetailsService.loadUserByUsername(
            rotation.user().getUsername()
      );
      String accessToken = jwtService.generateToken(userDetails);
return ResponseEntity.ok(
            new AuthResponse(accessToken, rotation.refreshToken())
      );
   }
   @PostMapping("/logout")     public
ResponseEntity<Void> logout(
         @Valid @RequestBody RefreshRequest request) {
refreshTokenService.revoke(request.refreshToken());
      return ResponseEntity.noContent().build();
   }
}
```

Create AesEncryptionService.java

```
package com.example.jwt_security_api.security; import
io.jsonwebtoken.io.Decoders; import
org.springframework.beans.factory.annotation.Value; import
org.springframework.stereotype.Service; import
javax.crypto.Cipher; import
javax.crypto.spec.GCMParameterSpec; import
javax.crypto.spec.SecretKeySpec; import
java.nio.charset.StandardCharsets; import
java.security.GeneralSecurityException; import
java.security.SecureRandom; import java.util.Base64;
@Service
public class AesEncryptionService {     private static final
int NONCE_LENGTH_BYTES = 12;     private static final
int TAG_LENGTH_BITS = 128;
   private final SecretKeySpec key;
   private final SecureRandom secureRandom = new SecureRandom();
   public AesEncryptionService(
         @Value("${security.encryption.aes-key-base64}") String keyBase64) {
byte[] keyBytes = Decoders.BASE64.decode(keyBase64);
```

```
        if (keyBytes.length != 32) {
throw new IllegalArgumentException(
                "AES key must decode to exactly 32 bytes"
            );
        }
        this.key = new SecretKeySpec(keyBytes, "AES");
    }
    public String encrypt(String plaintext) {
        try {
            byte[] nonce = new byte[NONCE_LENGTH_BYTES];
            secureRandom.nextBytes(nonce);
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(
                    Cipher.ENCRYPT_MODE,
                    key,
                    new GCMParameterSpec(TAG_LENGTH_BITS, nonce)
            );
            byte[] ciphertext = cipher.doFinal(
                    plaintext.getBytes(StandardCharsets.UTF_8)
            );
            return "v1:"
                    + Base64.getEncoder().encodeToString(nonce)
                    + ":"
                    + Base64.getEncoder().encodeToString(ciphertext);
        } catch (GeneralSecurityException e) {
            throw new IllegalStateException("Could not encrypt sensitive data", e);
        }
    }
    public String decrypt(String encryptedValue) {
    String[] parts = encryptedValue.split(":", 3);     if
    (parts.length != 3 || !"v1".equals(parts[0])) {
            throw new IllegalArgumentException("Unsupported encrypted value format");
        }
try {
            byte[] nonce = Base64.getDecoder().decode(parts[1]);
byte[] ciphertext = Base64.getDecoder().decode(parts[2]);            if
(nonce.length != NONCE_LENGTH_BYTES) {
                throw new IllegalArgumentException("Invalid AES-GCM nonce");
            }
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(
                    Cipher.DECRYPT_MODE,
                    key,
                    new GCMParameterSpec(TAG_LENGTH_BITS, nonce)
            );
            return new String(cipher.doFinal(ciphertext), StandardCharsets.UTF_8);
```

```
        } catch (GeneralSecurityException e) {
throw new IllegalStateException(
                "Could not decrypt sensitive data; it may have been altered",
e
            );
        }
    }}
```

Create StoredCredential.java(entity) package

```
com.example.jwt_security_api.entity;

import jakarta.persistence.*;
@Entity
@Table(name = "stored_credentials",
uniqueConstraints = {
            @UniqueConstraint(columnNames = {"user_id","provider"})
        }
)
public class StoredCredential {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id", nullable = false)
    private User user;
    @Column(nullable = false, length = 100)
private String provider;
    @Column(nullable = false, length = 4000)
private String encryptedSecret;
    protected StoredCredential() {
    }
    public StoredCredential(User user, String provider, String encryptedSecret) {
    this.user = user;     this.provider = provider;
        this.encryptedSecret = encryptedSecret;
    }     public Long
getId() {         return id;
    }
    public User getUser() {
        return user;
    }
    public String getProvider() {
        return provider;
    }
    public String getEncryptedSecret() {
        return encryptedSecret;
    }
    public void setEncryptedSecret(String encryptedSecret) {
this.encryptedSecret = encryptedSecret;
```

```
    }
}
```

Create StoredCredentialRepository.java: enforce one credential per user/provider

```
package com.example.jwt_security_api.repository; import
com.example.jwt_security_api.entity.StoredCredential; import
org.springframework.data.jpa.repository.JpaRepository; import java.util.Optional;
public interface StoredCredentialRepository extends JpaRepository<StoredCredential, Long> {
Optional<StoredCredential> findByUserUsernameAndProvider(
        String username,
        String provider
    ); }
```

Create CredentialService.java for storage and retrieval, encrypting before persistence and decrypting when the backend needs the secret

```
package com.example.jwt_security_api.service;
import com.example.jwt_security_api.entity.StoredCredential; import
com.example.jwt_security_api.entity.User;
import com.example.jwt_security_api.repository.StoredCredentialRepository;
import com.example.jwt_security_api.repository.UserRepository; import
com.example.jwt_security_api.security.AesEncryptionService; import
jakarta.persistence.EntityNotFoundException; import
org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
@Service
public class CredentialService { private final
    StoredCredentialRepository credentialRepository; private final
    UserRepository userRepository;
    private final AesEncryptionService encryptionService; public
    CredentialService(
            StoredCredentialRepository credentialRepository,
            UserRepository userRepository,
            AesEncryptionService encryptionService) {
this.credentialRepository = credentialRepository;        this.userRepository
= userRepository;
        this.encryptionService = encryptionService;
    }
    @Transactional
    public void storeSecret(String username, String provider, String secret) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new EntityNotFoundException("User not found"));
        String encryptedSecret = encryptionService.encrypt(secret);
        StoredCredential credential = credentialRepository
                .findByUserUsernameAndProvider(username, provider)
                .orElseGet(() -> new StoredCredential(user, provider, encryptedSecret));
credential.setEncryptedSecret(encryptedSecret);
        credentialRepository.save(credential);
    }
```

```
    @Transactional
    public String getSecretForBackendUse(String username, String provider) {
        StoredCredential credential = credentialRepository
                .findByUserUsernameAndProvider(username, provider)
                .orElseThrow(() -> new EntityNotFoundException(
                        "Credential not found"
                ));
        return encryptionService.decrypt(credential.getEncryptedSecret());
    }
}
```

7) OUTPUT

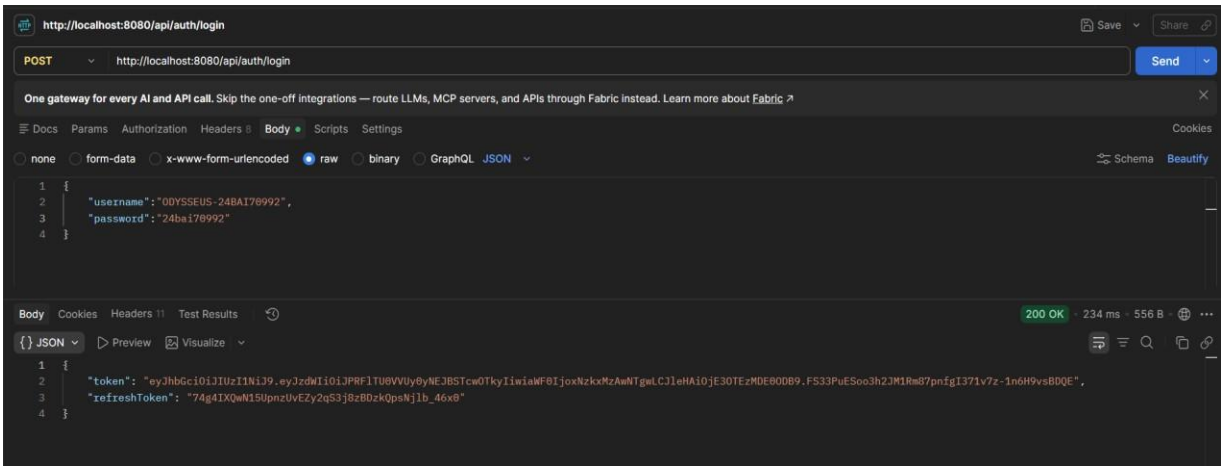


Fig7.2.1:A user logins in then token and refresh token are generated

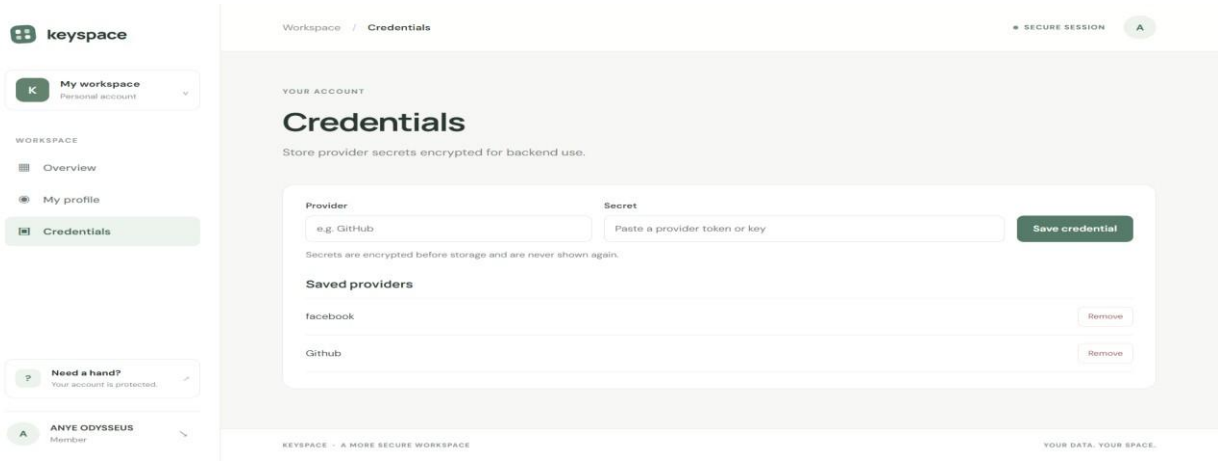


Fig 7.2.2: Stored credentials for user anye odysseus(AES encryption)

	id	encrypted_secret	provider	user_id
▶	2	v1:TLO0xE43d9DzrRkO:GjGN/tmbyLxatxVwfOs...	facebook	7
	3	v1:N3vhBRJQqZN9Jiip:u2pri1+29PNrM8Pv62Z4...	Github	7
*	NULL	NULL	NULL	NULL

Fig 7.2.3: Stored credentials in database

	id	expires_at	revoked	token_hash	user_id
▶	1	2026-10-09 15:38:44.769079	0	8e9a19db2e3590e345f755dd86a8e706fe4b760...	6
	2	2026-10-09 15:41:23.311701	1	d4d68a8bd57fa1bd23a4d6ed72719642479c95...	6
	3	2026-10-09 15:42:45.802439	0	2e20c18fd7d523e24383ca52053be637868e9ac...	6
	4	2026-10-09 16:18:10.677469	1	960f5210fccb83df68af3a328af7958309771bc1...	3
	5	2026-10-09 16:18:11.234529	1	b05ce9a8baae10fb2af30eb84d96ccd38c07f804...	3
	6	2026-10-09 16:19:44.300469	1	b7116abb94a98bd7b474e2f8d27f7b3835db332...	3
	7	2026-10-09 16:21:14.279807	0	1cbe6613f447ac7d83a4c088988e925f6abdcef4...	5
	8	2026-10-11 08:53:26.363779	1	f39e60b7260b433e0f60a9e01a14a69f43a8881...	5
	9	2026-10-11 08:56:53.431859	1	0b23c54c9cbcdc7b96957856 18544cbb22512ad...	3
	10	2026-10-11 08:56:53.578419	1	fecb668457cfd962124284663f5cf159f2b978ef...	3
	11	2026-10-13 08:25:07.069137	1	b09f11c1aa9178437ab90be1bb171ce550ae795...	3
	12	2026-10-13 08:25:07.608077	1	c1bc38d5a58eec5d10a3a7144835b6fc01e7b4d...	3
	13	2026-10-13 08:25:12.389055	1	d0a4be65cfb6d73324865825c739cb2c9828e2b...	3
	14	2026-10-13 08:25:15.815035	1	ffa60dd80eba6e7d38da78ce65e3258d2cf37e1...	3
	15	2026-10-13 08:25:19.452468	1	668d3cfba623359c4a2dc023c4ab863755c957e...	3
	16	2026-10-13 08:28:49.170093	1	216409352bcf01fabdc830aa7d256ea80b38255...	5
	17	2026-10-13 08:38:48.817253	1	254ae71ccaea9ff08cd8acf8bd42f07dca9b9785...	5

Fig 7.2.4: Refresh tokens for different users with user_ids in database

8) LEARNING OUTCOMES

- Explain the role of short-lived access tokens and longer-lived refresh tokens in an authentication system.
- Generate secure random refresh tokens and store only their SHA-256 hashes in the database.
- Implement refresh-token expiration, revocation, and rotation, including protection against concurrent reuse.
- Connect refresh and logout workflows to REST endpoints and return appropriate responses for invalid or expired tokens.
- Use AES-GCM to encrypt and decrypt sensitive credentials that the backend must recover later.
- Manage signing and encryption keys through external configuration rather than hard-coding them in source code.
- Distinguish hashed secrets, such as user passwords and refresh tokens, from recoverable encrypted data, and identify which service or API surfaces still need implementation.